

Module 06: Learning — Updating Dirichlet Concentrations

Learning Objectives

1. Implement parameter learning using Dirichlet concentration updates for the A and B matrices.
2. Compute expected parameter matrices from Dirichlet posteriors and compare with ground truth.
3. Evaluate model quality using Bayesian Model Reduction (BMR).

Introduction

In Modules 01–05, the agent's generative model was fixed — the A, B, C, D, E matrices were set once and never changed. But a truly adaptive agent must **learn**: it should update its beliefs about the environment's causal structure based on experience. In Active Inference, learning means updating the **Dirichlet concentration parameters** (p_A , p_B) that parameterize the agent's beliefs about A and B.

Key Concepts

1. Why Dirichlet Distributions?

Each column of the A-matrix is a categorical distribution. The Bayesian conjugate prior for a categorical distribution is the **Dirichlet distribution**. Instead of storing point estimates for A, the agent maintains concentration parameters $\mathbf{p}_A$ where:

$$P(\mathbf{A}[:, s]) = \text{Dir}(\mathbf{p}_A[:, s])$$

The expected (mean) A-matrix is:

$$\mathbb{E}[\mathbf{A}[:, s]] = \frac{\mathbf{p}_A[:, s]}{\sum_o \mathbf{p}_A[o, s]}$$

```

from active_inference.math import expected_A

pA = np.array([[10.0, 1.0],
               [1.0, 10.0]])

A_expected = expected_A(pA)
# A_expected ≈ [[0.909, 0.091], [0.091, 0.909]]

```

2. Updating pA: Learning the Likelihood

After each observation-state pair (o_t, q_s) , the agent updates pA:

$$\mathbf{p}_A[o_t, :] \mathrel{+} = q(s) \cdot \eta$$

where η is the learning rate. This is an outer-product update weighted by the posterior beliefs.

```

from active_inference.math import update_dirichlet_A

pA_new = update_dirichlet_A(
    pA=pA.copy(),
    observation=0,
    q_s=np.array([0.9, 0.1]),
    learning_rate=1.0,
)
# pA_new[0, 0] increased by 0.9 (matched obs-state pair)
# pA_new[0, 1] increased by 0.1 (secondary belief)

```

3. Updating pB: Learning the Transitions

Similarly, the transition concentrations are updated after each (state, action, next-state) triple:

$$\mathbf{p}_B[:, s, a] \mathrel{+} = q(s') \cdot q(s) \cdot \eta$$

```

from active_inference.math import update_dirichlet_B

pB_new = update_dirichlet_B(
    pB=pB.copy(),
    q_s=np.array([0.9, 0.1]),
    q_s_next=np.array([0.1, 0.9]),
    action=1,
    learning_rate=1.0,
)

```

4. Online Learning Loop

The full perception-action-learning loop at each timestep:

```

for t in range(100):
    # 1. Perceive
    action = agent.step(obs)

    # 2. Learn A
    pA = update_dirichlet_A(pA, obs, agent.q_s, learning_rate=1.0)
    agent.model.A = expected_A(pA)

    # 3. Act and observe
    obs = env.step(action)

    # 4. Learn B
    pB = update_dirichlet_B(pB, q_s_prev, agent.q_s, action, learning_rate=1.0)
    agent.model.B = expected_B(pB)

```

5. Tracking Learning Progress

Use KL divergence to measure how close the learned model is to the truth:

```

from active_inference.math import kl_divergence

# Compare each column of learned A vs true A
for s in range(num_states):
    kl = kl_divergence(expected_A(pA[:, s], true_A[:, s])
    print(f"KL[learned || true] for state {s}: {kl:.6f}")

```

Visualize with `plot_learning_progress()`:

```

from active_inference.visualization import plot_learning_progress
plot_learning_progress(kl_history)

```

6. Dirichlet Entropy

The entropy of the Dirichlet distribution measures the agent's uncertainty about a parameter column:

$$H[\text{Dir}(\boldsymbol{\alpha})] = \ln B(\boldsymbol{\alpha}) + (\alpha_0 - K) \psi(\alpha_0) - \sum_k (\alpha_k - 1) \psi(\alpha_k)$$

```

from active_inference.math import dirichlet_entropy

# Low entropy = confident about the parameter
H = dirichlet_entropy(pA[:, 0])
print(f"Entropy of pA column 0: {H:.4f}")

```

7. Bayesian Model Reduction (BMR)

BMR compares a full model against a reduced (simpler) model by computing the change in free energy:

$$\Delta F = F_{\text{reduced}} - F_{\text{full}}$$

A negative ΔF means the reduced model is better (simpler and equally good).

```
from active_inference.math import bayesian_model_reduction

delta_F = bayesian_model_reduction(pA_full, pA_reduced)
print(f" $\Delta F$  = {delta_F:.4f}")  # Negative = reduced model preferred
```

Applications

- **Multi-episode training:** Run many episodes, accumulating pA and pB across episodes. The agent develops increasingly accurate models.
- **Catastrophic forgetting:** Monitor whether learning in one context degrades performance in another.
- **Model comparison:** Use BMR to prune unnecessary parameters from the model.

Conclusion

Learning closes the loop: the agent not only perceives and acts but also improves its generative model over time. Dirichlet updates provide a principled Bayesian mechanism for this improvement. Module 07 extends these ideas to multi-agent settings where agents learn about each other.